

MSBA 265 - Foundational Module 1: Exploratory Data Analysis & Visual Outlier Audits

Author: Visiting Instructor Shyla Solis
Course: MSBA 265 - Business Analytics Topics
Dataset: French Motor Third Party Liability
Claims (OpenML ID: 41214)
Student: Min Yang

Business Problem Overview

In property and casualty insurance, policy pricing depends on understanding risk exposure, driver age, and population density. Before building predictive loss models or business strategy dashboards, we must inspect raw feature data types, construct a domain-specific Business Data Dictionary, plot feature distributions, evaluate univariate skewness, and visually diagnose extreme outliers.

```
In [1]: import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [2]: # Set visual styling for clean presentation graphics
plt.style.use("seaborn-v0_8-whitegrid")
plt.rcParams["font.sans-serif"] = "DejaVu Sans"
plt.rcParams["font.size"] = 10
pd.set_option("display.max_columns", 20)

# Load dataset
data_path = os.path.join(".", "data", "raw_business_data.csv")
if not os.path.exists(data_path):
    data_path = os.path.join("data", "raw_business_data.csv")
```

```
In [3]: df = pd.read_csv(data_path)
print(f"[+] Loaded Dataset Shape: {df.shape[0]:,} rows x {df.shape[1]} columns")
```

[+] Loaded Dataset Shape: 678,013 rows x 12 columns

Dataset Overview

The dataset contains 678,013 automobile insurance policy records and 12 features. Each row represents a policy record. The variables describe claim frequency, policy exposure, driver characteristics, vehicle characteristics, and geographic information.

The first ten records are displayed below to confirm that the dataset was loaded correctly and to provide an initial view of its structure.

```
In [4]: # Render interactive HTML table head
df.head(10)
```

Out[4]:

	IDpol	ClaimNb	Exposure	Area	VehPower	VehAge	DrivAge	BonusMalus	VehB
0	1.0	1	0.10	'D'	5	0	55	50	
1	3.0	1	0.77	'D'	5	0	55	50	
2	5.0	1	0.75	'B'	6	2	52	50	
3	10.0	1	0.09	'B'	7	0	46	50	
4	11.0	1	0.84	'B'	7	0	46	50	
5	13.0	1	0.52	'E'	6	2	38	50	
6	15.0	1	0.45	'E'	6	2	38	50	
7	17.0	1	0.27	'C'	7	0	33	68	
8	18.0	1	0.71	'C'	7	0	33	68	
9	21.0	1	0.15	'B'	7	0	41	50	

```
In [5]: print("=====")
print("          1. STRUCTURAL DATA TYPES & NULLS          ")
print("=====")
df.info()
```

```
=====
          1. STRUCTURAL DATA TYPES & NULLS
=====
<class 'pandas.DataFrame'>
RangeIndex: 678013 entries, 0 to 678012
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  -
0   IDpol            678013 non-null float64
1   ClaimNb          678013 non-null int64
2   Exposure         678013 non-null float64
3   Area             678013 non-null str
4   VehPower         678013 non-null int64
5   VehAge           678013 non-null int64
6   DrivAge          678013 non-null int64
7   BonusMalus       678013 non-null int64
8   VehBrand         678013 non-null str
9   VehGas           678013 non-null str
10  Density          678013 non-null int64
11  Region           678013 non-null str
dtypes: float64(2), int64(6), str(4)
memory usage: 62.1 MB
```

```
In [6]: print("\n=====")
print("          2. SUMMARY STATISTICS (NUMERICAL)          ")
print("=====")
df.describe().T
```

=====

2. SUMMARY STATISTICS (NUMERICAL)

=====

	count	mean	std	min	25%	50%
IDpol	678013.0	2.621857e+06	1.641783e+06	1.000000	1157951.00	2272152.00
ClaimNb	678013.0	5.324677e-02	2.401173e-01	0.000000	0.00	0.00
Exposure	678013.0	5.287501e-01	3.644415e-01	0.002732	0.18	0.00
VehPower	678013.0	6.454631e+00	2.050906e+00	4.000000	5.00	6.00
VehAge	678013.0	7.044265e+00	5.666232e+00	0.000000	2.00	6.00
DrivAge	678013.0	4.549912e+01	1.413744e+01	18.000000	34.00	44.00
BonusMalus	678013.0	5.976150e+01	1.563666e+01	50.000000	50.00	50.00
Density	678013.0	1.792422e+03	3.958647e+03	1.000000	92.00	395.00

```
In [7]: # Part 2.2: Raw Boundary Audit

audit_columns = ["DrivAge", "Exposure", "BonusMalus"]

boundary_audit = pd.DataFrame({
    "Minimum": df[audit_columns].min(),
    "Maximum": df[audit_columns].max(),
    "Negative Values": (df[audit_columns] < 0).sum(),
    "Zero Values": (df[audit_columns] == 0).sum(),
    "Missing Values": df[audit_columns].isna().sum()
})

display(boundary_audit)

# Check common numeric sentinel codes used to represent missing values
sentinel_codes = [-1, -9, -99, -999]

sentinel_audit = pd.DataFrame({
    code: (df[audit_columns] == code).sum()
    for code in sentinel_codes
}).T

sentinel_audit.index.name = "Sentinel Code"
display(sentinel_audit)
```

	Minimum	Maximum	Negative Values	Zero Values	Missing Values
DrivAge	18.000000	100.00	0	0	0
Exposure	0.002732	2.01	0	0	0
BonusMalus	50.000000	230.00	0	0	0

	DrivAge	Exposure	BonusMalus
Sentinel Code			
-1	0	0	0
-9	0	0	0
-99	0	0	0
-999	0	0	0

Data Quality Audit

The structural audit confirms that all 12 columns contain 678,013 non-null observations. Therefore, no standard missing values were detected. However, zero null values do not automatically prove that the data is fully clean because invalid values may be stored as ordinary numbers or sentinel codes.

The raw boundary audit found that **DrivAge** ranges from 18 to 100 years. It contains no negative values, zeros, missing values, or common sentinel codes. Therefore, no negative-age corruption or child-policyholder records were detected.

Exposure ranges from 0.002732 to 2.01. It contains no negative values, zeros, missing values, or common sentinel codes. Values above 1.0 should receive additional domain review because exposure commonly represents policy duration in years. However, these records are retained because the current audit does not provide evidence that they are data-entry errors.

BonusMalus ranges from 50 to 230. It contains no negative values, zeros, missing values, or common sentinel codes. High values may represent legitimate higher-risk or surcharge policyholders and should not be removed automatically.

VehAge ranges from 0 to 100 years. Its maximum requires additional domain validation because it may represent an antique vehicle, a capped value, or a special code.

IDpol is stored as a floating-point value, but it is an identifier rather than a continuous numerical feature. It should not be treated as a predictor in statistical modeling.

```
In [8]: business_definitions = {
    "IDpol": (
        "Policy ID",
        "Key ID",
        "Unique policy identification key assigned to contract."
    ),
    "ClaimNb": (
        "Claim Count",
        "Count",
        "Total insurance claims filed during policy coverage period."
    ),
    "Exposure": (
```

```

        "Risk Exposure",
        "Policy-years",
        "Amount of insured time represented by the policy record; used as an offset
    ),
    "Area": (
        "Area Risk Code",
        "Ordinal Category",
        "Geographic risk code of policyholder address (A = rural, F = urban).",
    ),
    "VehPower": (
        "Engine Power",
        "Tax HP",
        "Fiscal power rating category of vehicle engine."
    ),
    "VehAge": (
        "Vehicle Age",
        "Years",
        "Age of the insured vehicle in years."
    ),
    "DrivAge": (
        "Driver Age",
        "Years",
        "Age of the primary insured driver in years."
    ),
    "BonusMalus": (
        "Claim History Index",
        "Index Score (observed 50-230)",
        "Driver risk score in which lower values generally indicate a bonus and high
    ),
    "VehBrand": (
        "Brand Class",
        "Category Code",
        "Anonymized vehicle manufacturer/brand classification code."
    ),
    "VehGas": (
        "Fuel Type",
        "Binary Class",
        "Engine fuel type used (Diesel vs. Regular/Gasoline)."
    ),
    "Density": (
        "Population Density",
        "People / km2",
        "Population density of policyholder's municipality."
    ),
    "Region": (
        "Administrative Region",
        "Region Code",
        "French administrative region code corresponding to residency location."
    )
}

dict_list = []
for col in df.columns:
    label, unit, definition = business_definitions.get(
        col,
        (col, "N/A", "No business definition provided.")
    )
    dict_list.append({
        "Feature Name": col,
        "Business Label": label,

```

```
        "Data Type": str(df[col].dtype),
        "Unit": unit,
        "Null Count": df[col].isnull().sum(),
        "Unique Values": df[col].nunique(),
        "Business Definition": definition
    })

dict_df = pd.DataFrame(dict_list)
display(dict_df)

reports_dir = os.path.join(".", "reports")
os.makedirs(reports_dir, exist_ok=True)
dict_path = os.path.join(reports_dir, "data_dictionary.csv")
dict_df.to_csv(dict_path, index=False)
print(f"\n[+] Successfully exported Business Data Dictionary artifact to: {dict_
```

	Feature Name	Business Label	Data Type	Unit	Null Count	Unique Values	Business Definition
0	IDpol	Policy ID	float64	Key ID	0	678013	Unique policy identification key assigned to c...
1	ClaimNb	Claim Count	int64	Count	0	11	Total insurance claims filed during policy cov...
2	Exposure	Risk Exposure	float64	Policy-years	0	181	Amount of insured time represented by the poli...
3	Area	Area Risk Code	str	Ordinal Category	0	6	Geographic risk code of policyholder address (...)
4	VehPower	Engine Power	int64	Tax HP	0	12	Fiscal power rating category of vehicle engine.
5	VehAge	Vehicle Age	int64	Years	0	78	Age of the insured vehicle in years.
6	DrivAge	Driver Age	int64	Years	0	83	Age of the primary insured driver in years.
7	BonusMalus	Claim History Index	int64	Index Score (observed 50-230)	0	115	Driver risk score in which lower values genera...
8	VehBrand	Brand Class	str	Category Code	0	11	Anonymized vehicle manufacturer/brand classifi...
9	VehGas	Fuel Type	str	Binary Class	0	2	Engine fuel type used (Diesel vs. Regular/Gaso...
10	Density	Population Density	int64	People / km2	0	1607	Population density of policyholder's municipal...
11	Region	Administrative Region	str	Region Code	0	22	French administrative region code correspondin...

◀  ▶

[+] Successfully exported Business Data Dictionary artifact to: ..\reports\data_dictionary.csv

Business Data Dictionary Interpretation

The Business Data Dictionary connects each technical column name to its operational meaning, data type, unit, and business definition.

This step is important because variables with similar technical data types may require different analytical treatment. For example, `IDpol` is an identifier, `ClaimNb` is a discrete event count, `Exposure` represents policy duration, and `Density` is a continuous geographic measure.

The completed data dictionary was exported to:

```
reports/data_dictionary.csv
```

Modeling Defense: Exposure, Density, and Area

`Exposure` represents the amount of time that each policy was insured and at risk of producing a claim. A policy observed for a longer period has more opportunity to generate claims, so claim counts cannot be compared fairly without controlling for exposure time. In a Poisson claim-frequency model, `log(Exposure)` should be used as an offset with a fixed coefficient of one, allowing the model to estimate a claim rate rather than only a raw claim count. `Density` and `Area` both represent geographic risk information: `Density` is a continuous population measure, while `Area` is a categorical geographic risk class. For this linear modeling task, they should not be entered together because their information overlaps and may create redundancy or multicollinearity. The analyst should select one representation based on the modeling goal; `Density` provides more detailed numerical information, while dummy-encoded `Area` provides easier categorical interpretation.

```
In [9]: print("=====")
print("          UNIVARIATE SKEWNESS DIAGNOSTICS          ")
print("=====")

numeric_df = df.select_dtypes(include=[np.number])
skewness_series = numeric_df.skew().sort_values(ascending=False)

skew_summary = pd.DataFrame({
    "Feature": skewness_series.index,
    "Skewness Coefficient": skewness_series.values,
    "Diagnosis": [
        "Extremely Right-Skewed"
        if s > 3.0
        else "Moderately Skewed"
        if abs(s) > 1.0
        else "Unskewed / Symmetric"
        for s in skewness_series.values
    ],
    "Action Required": [
        "Log / Box-Cox Transform"
        if abs(s) > 1.0
        else "Standard Scaler Sufficient"
        for s in skewness_series.values
    ]
})

display(skew_summary)
```

=====				
UNIVARIATE SKEWNESS DIAGNOSTICS				
=====				
	Feature	Skewness Coefficient	Diagnosis	Action Required
0	ClaimNb	5.599613	Extremely Right-Skewed	Log / Box-Cox Transform
1	Density	4.651421	Extremely Right-Skewed	Log / Box-Cox Transform
2	BonusMalus	1.728934	Moderately Skewed	Log / Box-Cox Transform
3	VehPower	1.171344	Moderately Skewed	Log / Box-Cox Transform
4	VehAge	1.147989	Moderately Skewed	Log / Box-Cox Transform
5	DrivAge	0.435759	Unskewed / Symmetric	Standard Scaler Sufficient
6	IDpol	0.237890	Unskewed / Symmetric	Standard Scaler Sufficient
7	Exposure	0.085318	Unskewed / Symmetric	Standard Scaler Sufficient

```
In [10]: # Quantify the zero-heavy structure of ClaimNb

claim_zero_count = (df["ClaimNb"] == 0).sum()
claim_positive_count = (df["ClaimNb"] > 0).sum()
claim_zero_percentage = claim_zero_count / len(df) * 100

print(f"Zero-claim records: {claim_zero_count:,}")
print(f"Positive-claim records: {claim_positive_count:,}")
print(f"Zero-claim percentage: {claim_zero_percentage:.2f}%")
print("Natural logarithm at zero: log(0) is undefined and approaches -infinity.")
```

Zero-claim records: 643,953
Positive-claim records: 34,060
Zero-claim percentage: 94.98%
Natural logarithm at zero: log(0) is undefined and approaches -infinity.

Skewness Diagnosis and Domain Override

`ClaimNb` and `Density` have the strongest positive skewness. The automated rule recommends a Log or Box-Cox transformation for both variables, but the same recommendation should not be applied without considering their business meaning.

`Density` is a non-negative continuous feature with a long right tail. A `log1p(Density)` transformation may reduce the influence of extremely high-density locations while retaining valid urban policyholders.

The automated transformation recommendation is overridden for `ClaimNb`. Its skewness coefficient is approximately 5.60, but 643,953 of the 678,013 records, or 94.98%, have exactly zero claims. A natural logarithm cannot be applied because $\log(0) = -\infty$, and a standard Box-Cox transformation also requires strictly positive values. Although `log1p()` is mathematically defined at zero, transforming the claim count would change the interpretation of the target and would not properly represent its discrete count structure. Therefore, `ClaimNb` should remain in raw count units and be modeled with a count-based method, such as Poisson or Negative Binomial regression, using `log(Exposure)` as an offset.

`IDpol` is excluded from transformation decisions because it is an identifier rather than a meaningful continuous predictor.

```
In [11]: print("=====")
print("          PEARSON MULTICOLLINEARITY AUDIT          ")
print("=====")

corr_matrix = numeric_df.corr().abs()

upper_tri = corr_matrix.where(
    np.triu(np.ones(corr_matrix.shape), k=1).astype(bool)
)

high_corr_pairs = []

for col in upper_tri.columns:
    high_corr_features = upper_tri.index[
        upper_tri[col] >= 0.85
    ].tolist()

    for feat in high_corr_features:
        val = upper_tri.loc[feat, col]
        high_corr_pairs.append({
            "Feature 1": feat,
            "Feature 2": col,
            "Pearson Correlation (|r|)": round(val, 4),
            "Status": "ACTION REQUIRED: Severe Multicollinearity"
        })

if high_corr_pairs:
    display(pd.DataFrame(high_corr_pairs))
else:
    print(
        "[+] No severe collinear feature pairs (|r| >= 0.85) "
        "detected across numerical metrics."
    )
```

```
=====
          PEARSON MULTICOLLINEARITY AUDIT
=====
[+] No severe collinear feature pairs (|r| >= 0.85) detected across numerical met
rics.
```

Collinearity Audit

No numerical feature pair reached the severe Pearson correlation threshold of $|r| \geq 0.85$. Therefore, the audit did not identify an immediate need to remove a numerical predictor because of severe pairwise linear correlation.

This result only evaluates pairwise Pearson correlation. It does not prove that all Variance Inflation Factors are low, and it does not capture nonlinear relationships.

In a formal predictor audit, `IDpo1` should be excluded because it is an identifier, and `ClaimNb` should be treated as the target rather than an input predictor.

```
In [12]: # Cell 6b: Render Annotated Pearson Correlation Heatmap
plt.figure(figsize=(10, 8))

# Compute Pearson correlation matrix across numerical variables
corr = numeric_df.corr()

# Generate a diverging color map (Cool-Warm: Blue = Negative, Red = Positive)
sns.heatmap(
    corr,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    vmin=-1.0,
    vmax=1.0,
    linewidths=0.5,
    cbar_kws={"label": "Pearson Correlation Coefficient (r)"}
)

plt.title(
    "Pearson Correlation Heatmap: Numerical Features",
    fontsize=14,
    fontweight="bold",
    pad=12
)
plt.tight_layout()

heatmap_path = os.path.join(
    "..", "reports", "figures", "correlation_heatmap.png"
)
plt.savefig(heatmap_path, dpi=300)
plt.show()

print(f"[+] Saved correlation heatmap graphics artifact to: {heatmap_path}")
```



[+] Saved correlation heatmap graphics artifact to: `..\reports\figures\correlation_heatmap.png`

Heatmap Interpretation

- Distribution Shape:** Most off-diagonal Pearson correlations are close to zero, indicating that the numerical features generally have weak pairwise linear relationships. No numerical feature pair reaches the severe correlation threshold of $|r| \geq 0.85$.
- Key Metrics / Relationships:** The strongest visible non-diagonal relationship is between `DrivAge` and `BonusMalus`, with approximately $r = -0.48$. This is a moderate negative association, meaning that older drivers tend to have lower BonusMalus scores in this dataset. It does not prove that age directly causes a lower score.
- Recommended Analytical Action:** No numerical predictor needs to be removed solely because of severe pairwise Pearson correlation. However, `IDpol` should be excluded because it is an identifier, and `ClaimNb` should be treated as the target rather than an input predictor. Variance Inflation Factors should be checked after all categorical variables have been encoded.

```
In [13]: fig, axes = plt.subplots(2, 2, figsize=(14, 10))

sns.boxplot(
    x=df["Density"],
```

```

        ax=axes[0, 0],
        color="lightcoral"
    )
    axes[0, 0].set_title(
        "Box Plot: Population Density (People / km2)",
        fontsize=12,
        fontweight="bold"
    )
    axes[0, 0].set_xlabel("Density")

    sns.histplot(
        df["DrivAge"],
        kde=True,
        ax=axes[0, 1],
        color="skyblue",
        bins=30
    )
    axes[0, 1].set_title(
        "Histogram & KDE: Driver Age Distribution",
        fontsize=12,
        fontweight="bold"
    )
    axes[0, 1].set_xlabel("Driver Age (Years)")

    sns.boxplot(
        x=df["BonusMalus"],
        ax=axes[1, 0],
        color="mediumpurple"
    )
    axes[1, 0].set_title(
        "Box Plot: BonusMalus Risk Index",
        fontsize=12,
        fontweight="bold"
    )
    axes[1, 0].set_xlabel("BonusMalus Index Score")

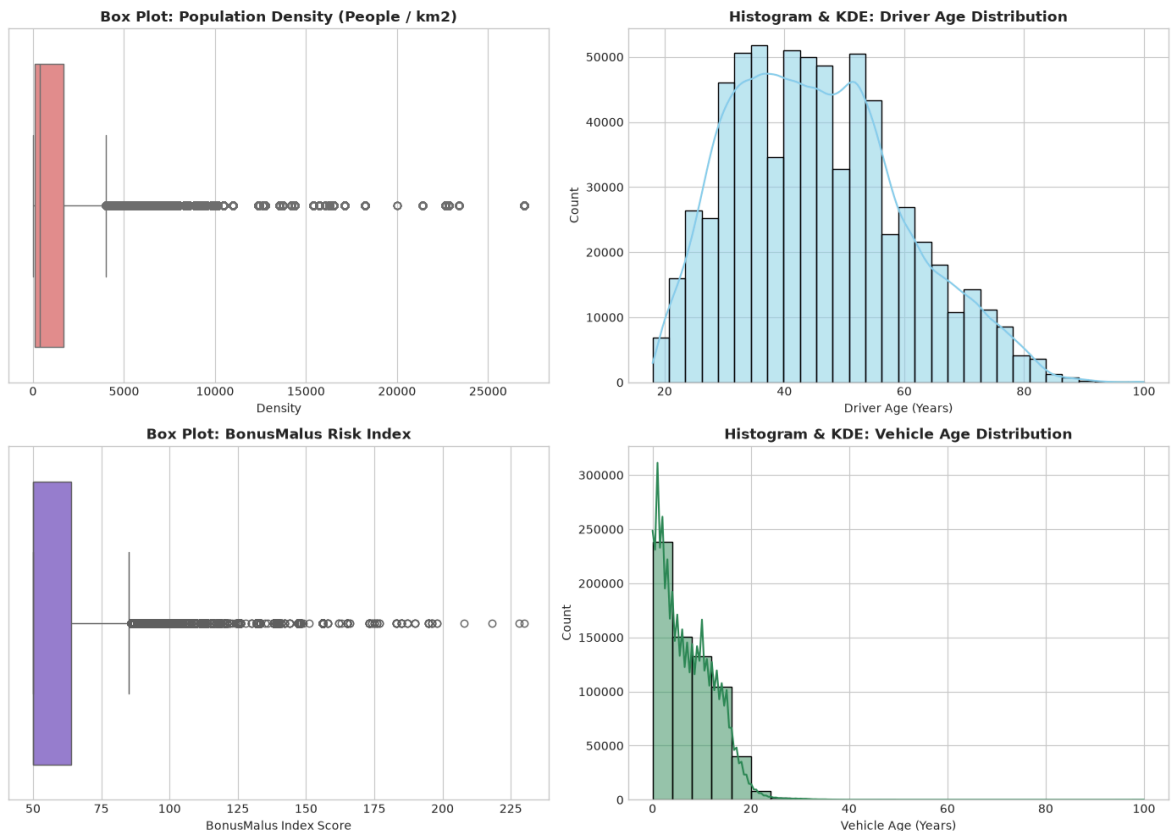
    sns.histplot(
        df["VehAge"],
        kde=True,
        ax=axes[1, 1],
        color="seagreen",
        bins=25
    )
    axes[1, 1].set_title(
        "Histogram & KDE: Vehicle Age Distribution",
        fontsize=12,
        fontweight="bold"
    )
    axes[1, 1].set_xlabel("Vehicle Age (Years)")

    plt.tight_layout()

    fig_path = os.path.join(
        "..", "reports", "figures", "feature_distributions.png"
    )
    plt.savefig(fig_path, dpi=300)
    plt.show()

    print(f"[+] Saved distribution graphics artifact to: {fig_path}")

```



[+] Saved distribution graphics artifact to: `..\reports\figures\feature_distributions.png`

Visual Distribution Audit

1. Population Density Box Plot

- **Distribution Shape:** `Density` is extremely right-skewed, with most observations concentrated at lower density levels and a long tail of high-density municipalities.
- **Key Metrics / Whiskers:** The first quartile is 92, the median is 393, and the third quartile is 1,658 people per square kilometer. The Tukey upper fence is 4,007, while the observed maximum is 27,000. Values above 4,007 appear as statistical outliers in the box plot.
- **Recommended Analytical Action:** The high-density observations may represent valid urban policyholders rather than data errors. They should not be removed automatically. A `log1p(Density)` transformation should be evaluated for models that are sensitive to strong right skewness.

2. Driver Age Histogram and KDE

- **Distribution Shape:** `DrivAge` is relatively balanced with mild positive skewness. Most policyholders are concentrated between approximately 30 and 60 years of age, followed by a gradual right tail.
- **Key Metrics / Whiskers:** The first quartile is 34, the median is 44, and the third quartile is 55 years. The observed range is 18 to 100 years, and the skewness

coefficient is approximately 0.44.

- **Recommended Analytical Action:** No logarithmic transformation or automatic outlier removal is required. Standard scaling may be considered when using a scale-sensitive model.

3. BonusMalus Box Plot

- **Distribution Shape:** `BonusMalus` is right-skewed and strongly concentrated near its lower boundary. A smaller number of high scores extend the right tail.
- **Key Metrics / Whiskers:** The first quartile and median are both 50, while the third quartile is 64. The Tukey upper fence is 85, and the observed maximum is 230. Values above 85 appear as statistical outliers.
- **Recommended Analytical Action:** High values may represent legitimate higher-risk or surcharge policyholders and should not be deleted automatically. Their business validity should be checked, and an interpretable indicator such as `BonusMalus > 100` may be considered.

4. Vehicle Age Histogram and KDE

- **Distribution Shape:** `VehAge` is positively skewed. Most vehicles are concentrated at lower ages, with a sparse right tail extending to 100 years.
- **Key Metrics / Whiskers:** The first quartile is 2, the median is 6, and the third quartile is 11 years. The Tukey upper fence is 24.5 years, and the observed maximum is 100.
- **Recommended Analytical Action:** Vehicle ages above 24.5 years should be flagged for additional domain validation. They should be retained, capped, or removed only after confirming whether they represent antique vehicles, capped values, or special codes.

```
In [19]: target_feature = "Density"

Q1 = df[target_feature].quantile(0.25)
Q3 = df[target_feature].quantile(0.75)
IQR = Q3 - Q1

iqr_lower = Q1 - 1.5 * IQR
iqr_upper = Q3 + 1.5 * IQR

df_iqr_cleaned = df[
    (df[target_feature] >= iqr_lower)
    & (df[target_feature] <= iqr_upper)
]

z_scores = (
    df[target_feature] - df[target_feature].mean()
) / df[target_feature].std()
```

```

df_z_cleaned = df[abs(z_scores) <= 3.0]

print(f"=== OUTLIER AUDIT REPORT FOR {target_feature} ===")
print(f"Original Record Count: {len(df):,} records")
print(
    f"Tukey IQR Range [{iqr_lower:.2f}, {iqr_upper:.2f}]: "
    f"Excludes {len(df) - len(df_iqr_cleaned):,} records"
)
print(
    f"Z-Score Threshold ( $|Z| \leq 3.0$ ): "
    f"Excludes {len(df) - len(df_z_cleaned):,} records"
)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.boxplot(
    x=df[target_feature],
    ax=axes[0],
    color="salmon"
)
axes[0].set_title(
    f"BEFORE Filtering: Raw {target_feature}",
    fontsize=12,
    fontweight="bold"
)
axes[0].set_xlabel("Population Density (People / km2)")

sns.boxplot(
    x=df_iqr_cleaned[target_feature],
    ax=axes[1],
    color="lightgreen"
)
axes[1].set_title(
    f"AFTER Filtering: IQR Tukey Filtered {target_feature}",
    fontsize=12,
    fontweight="bold"
)
axes[1].set_xlabel("Population Density (People / km2)")

plt.tight_layout()

fig_filter_path = os.path.join(
    "..",
    "reports",
    "figures",
    "outlier_filtering_comparison.png"
)

plt.savefig(fig_filter_path, dpi=300)
plt.show()

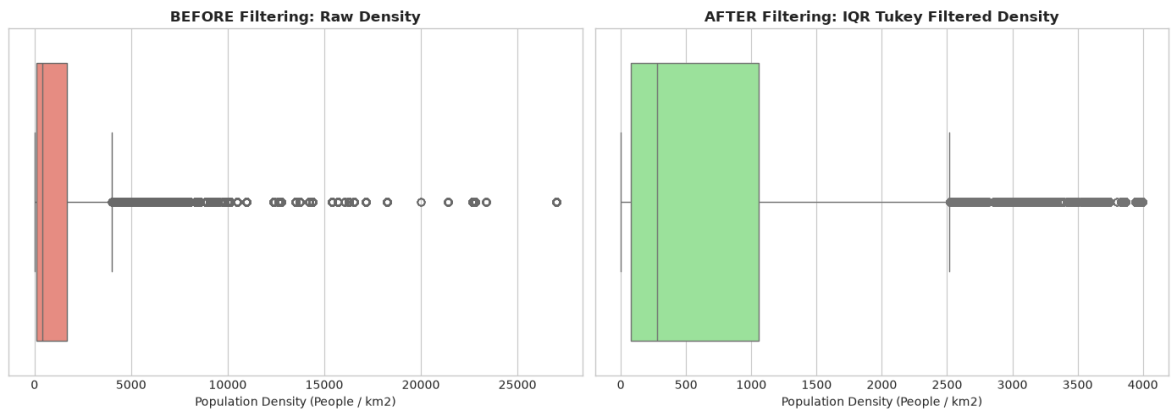
print(
    f"[+] Saved visual outlier comparison artifact to: "
    f"{fig_filter_path}"
)

```

```

=== OUTLIER AUDIT REPORT FOR Density ===
Original Record Count: 678,013 records
Tukey IQR Range [-2257.00, 4007.00]: Excludes 77,566 records
Z-Score Threshold ( $|Z| \leq 3.0$ ): Excludes 15,209 records

```



[+] Saved visual outlier comparison artifact to: `..\reports\figures\outlier_filtering_comparison.png`

Tukey IQR and Z-Score Outlier Comparison

Baseline Outlier Audit

The outlier audit evaluated `Density` across 678,013 policy records. The Tukey IQR method established a valid range from -2,257 to 4,007 people per square kilometer and flagged 77,566 records, or approximately 11.44%, as statistical outliers.

The Z-score method flagged 15,209 records, or approximately 2.24%. Therefore, the Tukey method identified 62,357 more observations than the Z-score method.

Methodological Interpretation

The two methods produce different results because `Density` is strongly right-skewed. Extreme metropolitan values increase both the mean and standard deviation, causing the parametric Z-score cutoff to move farther into the right tail.

The Tukey IQR method uses quartiles and is less affected by extreme values. It therefore identifies more high-density observations as statistical outliers.

Business Decision

Statistical outliers are not automatically data errors. Observations above 4,007 people per square kilometer may represent valid urban policyholders.

If the business objective is statewide or national insurance pricing, automatically removing 11.44% of the records could introduce geographic selection bias. In this scenario, the records should be retained and `log1p(Density)` should be evaluated.

If the downstream model is highly sensitive to leverage points and the analysis is intentionally limited to a lower-density population, Tukey filtering may be considered only with a documented business justification.

For tree-based models, mathematical outlier removal may be unnecessary because these models split features using decision thresholds and are generally less sensitive

to monotonic transformations.

```
In [20]: # Verify the descriptive statistics used in the outlier audit.  
# The observed results differ from the numerical example in the workshop.  
print(df["Density"].describe())  
print("Q1:", df["Density"].quantile(0.25))  
print("Q3:", df["Density"].quantile(0.75))  
print("Mean:", df["Density"].mean())  
print("Std:", df["Density"].std())  
print("Maximum:", df["Density"].max())
```

```
count      678013.000000  
mean        1792.422405  
std         3958.646564  
min           1.000000  
25%          92.000000  
50%         393.000000  
75%        1658.000000  
max        27000.000000  
Name: Density, dtype: float64  
Q1: 92.0  
Q3: 1658.0  
Mean: 1792.4224048801425  
Std: 3958.6465635595996  
Maximum: 27000
```

Verification Note

The observed statistics differ from the numerical example shown in the workshop document. The current analysis uses the values calculated directly from the downloaded dataset:

- Q1 = 92
- Median = 393
- Q3 = 1,658
- Mean = 1,792.42
- Standard deviation = 3,958.65
- Maximum = 27,000

The reason for the difference cannot be confirmed from the current evidence. Possible explanations include a different dataset version, rounded instructional examples, or a different calculation configuration. This notebook reports the reproducible results generated from the current data file.

Final Findings and Modeling Recommendations

The Module 1 audit produced the following conclusions:

1. The dataset contains 678,013 records and 12 variables with no standard null values.
2. `IDpo1` is an identifier and should not be used as a continuous predictor.

3. `ClaimNb` is a zero-heavy count target and should not receive an automatic logarithmic transformation.
4. `Exposure` represents policy duration and should be incorporated when modeling claim frequency.
5. `Density` is strongly right-skewed and is a candidate for `log1p` transformation.
6. `DrivAge` is relatively balanced and does not require a logarithmic transformation.
7. High `BonusMalus` values may represent legitimate surcharge customers and should not be deleted automatically.
8. Extremely high `VehAge` values require additional domain validation.
9. No severe pairwise numerical correlation was detected at $|r| \geq 0.85$.
10. Outlier treatment must depend on the business objective and downstream model rather than an automatic statistical rule.

Reproducible Outputs

This notebook generates the following artifacts:

- `reports/data_dictionary.csv`
- `reports/figures/correlation_heatmap.png`
- `reports/figures/feature_distributions.png`
- `reports/figures/outlier_filtering_comparison.png`

The production filtering script is stored in:

- `src/clean_outliers.py`